

8장_이미지 분류

이미지 분류 개요

이미지 분류: 이미지에서 객체(Object), 장면(Scenes) 등의 요소를 인식하고 분류하는 알고리즘

- 지도 학습
- 데이터셋 구성 = 이미지 + 이미지에 해당하는 클래스

이미지 분류 유형

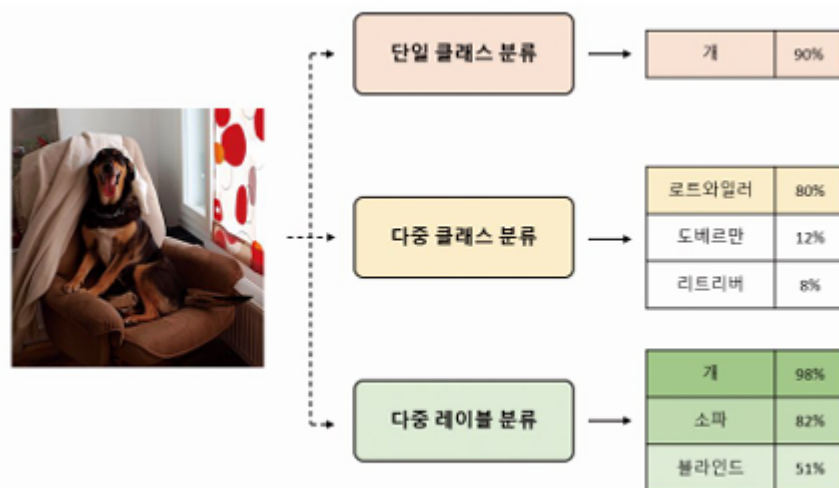


그림 8.1 이미지 분류

- 단일 클래스 분류(Single Classification)
 - 이미지를 하나의 대표 클래스로 분류 (이진 선택)
 - ex) 이미지 → “개” 이미지인가 아닌가?
- 다중 클래스 분류(Multi-Class Classification)
 - 개, 고양이 등 다중 클래스 중 이 이미지를 설명하는 클래스로 분류
- 다중 레이블 분류(Multi-Label Classification):
 - 입력 이미지에서 여러 클래스 예측
 - ex) 이미지 안에 있는 개, 소파, 블라인드 등 검출

이미지 분류 기술

- 서포트 벡터 머신(SVM)
- KNN
- Decision Tree
- 인공 신경망(ANN)
- 합성곱 신경망(CNN)

AlexNet

- 2012년 ILSVRC(ImageNet Large Scale Visual Recognition Challenge) 대회에서 우승한 CNN 모델
- 알렉스넷 이전 모델은 얇은 구조(Shallow Architecture)로 인식 오류율이 약 26% → 알렉스넷은 깊은 구조(Deep A) 약 16%로 감소
- 깊은 구조의 초기 모델

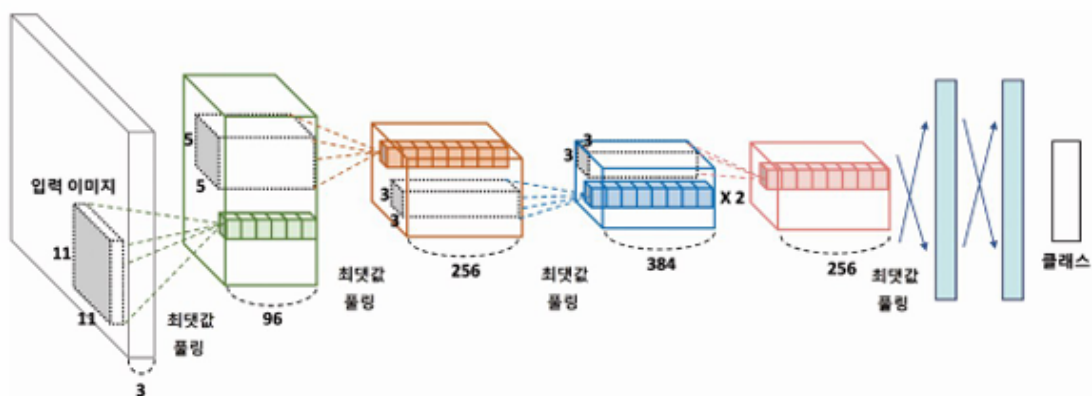


그림 8.2 알렉스넷 구조

- 이미지 특징 추출: 합성곱, 최대값 풀링 계층 활용
- 클래스 분류: 완전 연결 계층
- 순전파 과정에서 특징 맵 차원 수 증가, 크기 감소
 - 차원 수 증가: 모델의 표현력 증가
 - 특징 맵 크기 감소: 연산량 감소

LeNet-5와 AlexNet



LeNet-5 구조

- 1개 입력층 [I]: tanh
- 2개 합성곱 계층 [C1], [C3]: tanh
- 2개 서브샘플링 계층 [S2], [S4]: 평균값 풀링
- 1개 완전 연결 계층 [FC5]: tanh
- 1개 출력층 [O]: 시그모이드 활성화 함수

AlexNet 구조

- 1개 입력층
- 5개 합성곱 계층
- 3개 서브샘플링 계층 (+ 최댓값 풀링, ReLU)
- 3개 완전 연결 계층
- 1개 출력층

차이점

- 입력 이미지 크기 변화

- 활성화 함수
 - Rescaled

	LeNet-5	AlexNet
입력 이미지 크기		
활성화 함수	Rescaled Logistic Sigmoid Function - 기울기 소실 문제 유발	ReLU - 기울기 소실 문제 없는 비선형 활성화 함수 → 깊은 계층 가능
풀링 방식	평균값 풀링	최댓값 풀링 → 값의 분포가 더 일정해짐 ⇒ 기울기 계산이 더 쉬움
드롭아웃	X	O → 과적합 문제 완화

모델 학습

- 파이토치 torchinfo 라이브러리로 모델 구조 확인

```
# 모델 요약 함수
information = torchinfo.summary(model, input_data)
## 각 계층 출력 형태, 매개변수 수 확인 가능

# 모델 불러오기 함수
model = torchvision.models.alexnet(
    weights="AlexNet_Weights.IMAGENET1K_V1"
)
```

- 사전 학습된 가중치(weights) 매개변수 → 대규모 데이터셋에서 학습된 알렉스넷 로드

표 8.1 AlexNet_Weights,IMAGENET1K_V1 모델 정보

속성	값
acc@1	56.522
acc@5	79.066
입력 이미지 최소 크기	63×63
매개변수의 수	61,100,840
카테고리(클래스) 수	1,000
GFLOPS	0.71
파일 크기	233.1MB

- acc@n: 상위 n개 레이블에 대한 예측 정확도
- GFLOPS (Giga-Floating Point Operations Per Second): 해당 모델에 대한 컴퓨팅 성능
 - 모델이 1초 동안 수행할 수 있는 부동 소수점 산술 연산 수

표 8.2 알렉스넷에 사용된 계층 클래스 목록

계층	설명
시퀀셜(Sequential)	계층 인스턴스 묶음
합성곱(Conv2d)	3차원 텐서를 입력받아 합성곱 연산을 수행
ReLU(ReLU)	ReLU 활성화 수행
최대값 풀링(MaxPool2d)	3차원 텐서를 입력받아 최대값 풀링 수행
적응형 평균값 풀링(AdaptiveAvgPool2d)	입력 텐서의 형태에 상관없이 일정한 형태로 풀링 수행
드롭아웃(Dropout)	일정한 비율로 드롭아웃 수행
선형 변환(Linear)	2차원 텐서를 입력받는 완전 연결 계층

VGG

- 2014년 ILSVRC 준우승한 합성곱 신경망 모델
- 인식 오류율 약 7%

- 구글넷의 경우 인식 오류율이 약 6%로 더 낮고, 인셉션 모듈을 사용해 다양한 필터 크기와 풀링 작업으로 병렬 합성곱 연산 수행
 - 장점: 전역 특징과 지역 특징을 모두 포착해 성능 향상
 - 단점: 복잡한 구조 → 활용이 어려움
- 16개 계층 (13개 합성곱 계층 + 3개 완전 연결 계층)
- 작은 3x3 필터만으로 합성곱 연산 수행해 더 깊은 구조의 모델을 만들

AlexNet과 VGG-16

공통점

- 이미지넷 데이터로 학습
 - 알렉스넷 약 120만 개 데이터
 - VGG-16 약 1400만 개 데이터
- 모델 구조

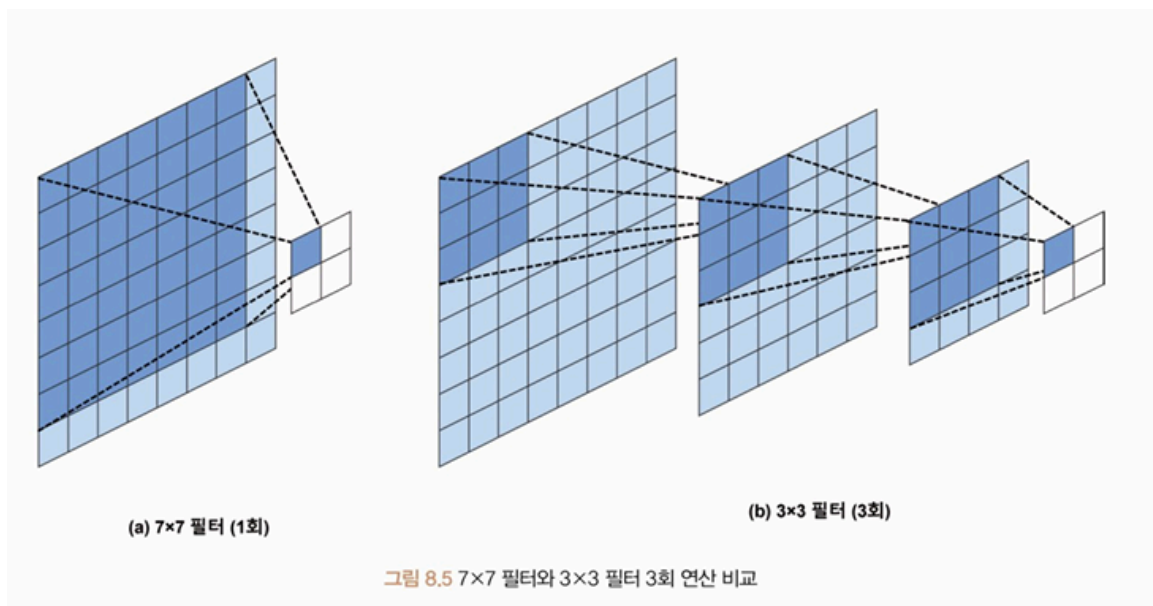


- 합성곱, ReLU, 풀링, 완전 연결 계층을 사용해 구조 설계

차이점

	AlexNet	VGG-16
합성곱 계층 크기	11×11, 5×5 → 수용 영역(RF) 확보 ⇒ 전역 특징 학습에 용이	3×3 → 이미지 특징을 더 정확히 분석하는 방법 (지역 특징 학습에 용이)
CNN으로 인한 차원 수 증가	512	384
인식 오류율	16%	7%

- 합성곱 계층 필터 크기에 따른 결과 비교 (7×7 필터 → 2×2 특징 맵)



작은 맵을 사용하면 필터를 여러 번 적용해야 같은 결과를 얻을 수 있음

- 특징 추출 효과 향상
- 비선형성 확보 ⇒ 모델 표현력 상승

실습

ResNet (Residual Network)

- 2015년 마이크로소프트 연구팀
- 합성곱 신경망 모델
- 인식 오류율 3.57%
- 대규모 이미지넷 데이터로 학습

- 구성: 합성곱, ReLU, 풀링, 완전 연결 계층

VGG 모델은 더 작은 크기의 필터를 사용해 계산 효율성을 향상시켰지만 깊은 신경망 구조로 인해 기울기 소실 문제가 발생했었음

→ ResNet: 잔차 연결(Residual Connection), 항등 사상(Identity Mapping), 잔차 블록(Residual Block) 등으로 기울기 소실 문제 해결, 계산 효율성 높임

- 152개 계층

ResNet 특징

- 기본 구조: 입력층, 합성곱, 배치 정규화, 활성화 함수, 잔차 블록, 평균값 풀링, 완전 연결, 출력층
- 34, 50, 101, 152개 계층으로 구성된 네트워크
- 모델은 잔차 블록의 개수에 따라 이름 결정

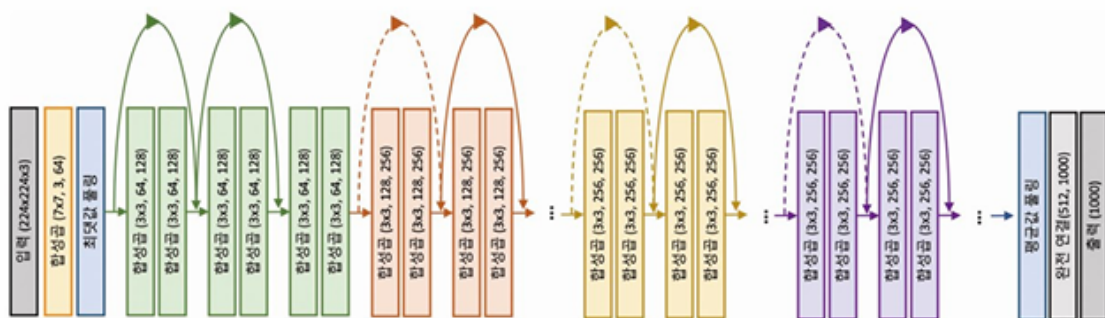


그림 8.6 ResNet-34 구조

- 34개 계층으로 구성된 모델
- 두 개의 합성곱 계층 + 단축 연결
 - 단축 연결: 이전 계층 출력값을 현재 계층의 입력값과 더해주는 방식으로 구현
 - 즉 이전 정보 → 다음 정보
 - 이전 계층 정보를 계속 전달하면서, 모델이 깊어져도 기울기 소실 문제 X, 즉 정보 손실 현상 X

기울기 저하 문제

- 깊은 계층 쌓기 실험
 - ResNet-20의 정확도 > ResNet-50의 정확도

- 기울기 저하 문제 (Degradation problem): 일정 수준 이상으로 계층을 깊게 쌓으면 오히려 학습되지 않는 현상
- 입력과 출력 사이의 차이만 학습해 기울기 저하 문제 해결

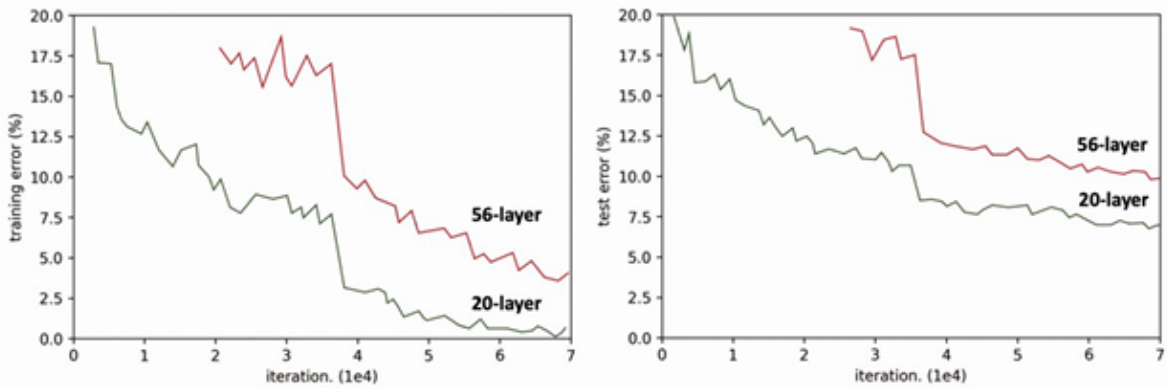


그림 8.7 계층 수에 따른 기울기 저하 현상

잔차 학습

- 항등 사상 실험
 - 계층이 깊어질수록 학습이 어렵다?
 - 얇은 모델을 먼저 학습하고 항등 사상으로 초기화된 계층을 추가해서 깊게 구성하자
 - 항등 사상: 입력값과 출력값이 동일한 연산
 - 즉 이미 학습된 모델의 결과를 그대로 출력하는 구조
- 입력과 출력이 동일함에도 기울기 저하 문제 발생
 - ⇒ 단순히 계층을 깊게만 구성하더라도 기울기가 저하된다
- 잔차 학습 기법을 적용해 문제 해결
 - 잔차 학습: 모델이 입력과 출력 사이의 차이(Residual)만 학습하게 하는 방법

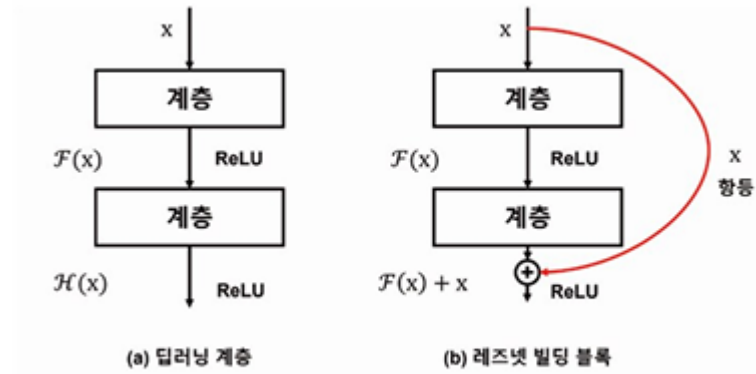


그림 8.8 잔차 학습

- 기존 ann 학습 방법: 이전 계층에서 활성화된 값을 다음 계층으로 바로 전달
 - 계층이 깊어질수록 $H(x)$ 최적화 어려움
- ResNet: $H(x) \rightarrow F(x) + x$ 변경
 - 빌딩 블록
 - x 는 항등 사상이므로 이미 정해진 고정값
 - 잔차 연결을 통해 입력값 x 가 출력값에 더해져 이전 계층에서 학습된 정보가 보존되는 것

잔차 연결

- 스킵 연결, 단축 연결
- 입력값이 신경망 계층을 통과한 후 출력값에 더해지는 연결
- 잔차 연결 수식

수식 8.1 동일한 차원 간 잔차 연결

$$y = \mathcal{F}(x, \{W_i\}) + x$$

- $\mathcal{F}$: 입력값 x 가 여러 계층을 통과한 결과값
- W_i : 현재 계층
- $\mathcal{F}$ 와 x 의 차원이 동일하면 덧셈 연산이 가능함
 - 동일하지 않을 경우?

수식 8.2 동일하지 않은 차원 간 잔차 연결

$$y = \mathcal{F}(x, \{W_i\}) + W_{sr}$$

병목 블록

- 깊은 구조 구성 → 연산량이 늘어나 학습이 어려움
- 깊은 구조 + 연산량 감소를 위해 병목 블록(Bottleneck Block) 추가

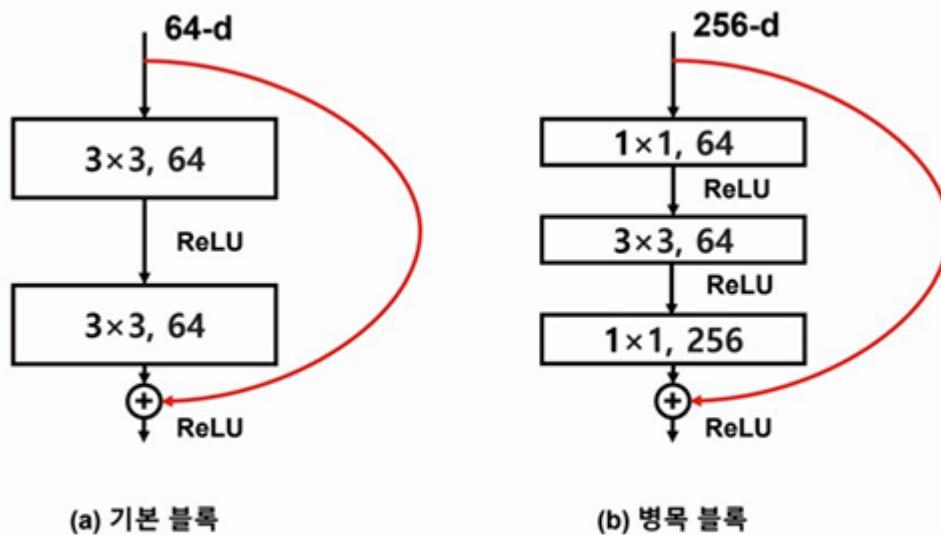


그림 8.9 기본 블록과 병목 블록

- 1x1 Conv → 입력 특징맵 차원 수를 줄임
- 3x3 Conv → 필터 적용
- 1x1 Con → 특징 맵 차원 수 증가

1x1 합성곱 계층을 통해 연산량이 줄고 모델 표현력이 유지됨

모델 구현

구조	출력 크기	ResNet-18	ResNet-34	ResNet-50	ResNet-101	ResNet-152
입력 줄기 (input stem)	112 × 112	7 × 7 64				
		stride=2, padding=3				
	56 × 56	7 × 7 MaxPool				
		stride=2, padding=1				
스테이지 1 (stage 1)	56 × 56	$\begin{bmatrix} 3 \times 3 & 64 \\ 3 \times 3 & 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3 & 64 \\ 3 \times 3 & 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 64 \\ 3 \times 3 & 64 \\ 1 \times 1 & 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 64 \\ 3 \times 3 & 64 \\ 1 \times 1 & 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 64 \\ 3 \times 3 & 64 \\ 1 \times 1 & 256 \end{bmatrix} \times 3$
스테이지 2 (stage 2)	28 × 28	$\begin{bmatrix} 3 \times 3 & 128 \\ 3 \times 3 & 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3 & 128 \\ 3 \times 3 & 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1 & 128 \\ 3 \times 3 & 128 \\ 1 \times 1 & 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1 & 128 \\ 3 \times 3 & 128 \\ 1 \times 1 & 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1 & 128 \\ 3 \times 3 & 128 \\ 1 \times 1 & 512 \end{bmatrix} \times 8$
스테이지 3 (stage 3)	14 × 14	$\begin{bmatrix} 3 \times 3 & 256 \\ 3 \times 3 & 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3 & 256 \\ 3 \times 3 & 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 & 256 \\ 3 \times 3 & 256 \\ 1 \times 1 & 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 & 256 \\ 3 \times 3 & 256 \\ 1 \times 1 & 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1 & 256 \\ 3 \times 3 & 256 \\ 1 \times 1 & 1024 \end{bmatrix} \times 36$
스테이지 4 (stage 4)	7 × 7	$\begin{bmatrix} 3 \times 3 & 512 \\ 3 \times 3 & 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3 & 512 \\ 3 \times 3 & 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 512 \\ 3 \times 3 & 512 \\ 1 \times 1 & 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 512 \\ 3 \times 3 & 512 \\ 1 \times 1 & 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 512 \\ 3 \times 3 & 512 \\ 1 \times 1 & 2048 \end{bmatrix} \times 3$
완전 연결 계층 (FC Layer)	1 × 1	1 × 1 AvgPool 1000 dimension(class)				
Params		11,689,512	21,797,672	25,557,032	44,549,160	60,192,808
GFLOPS		1.81	3.66	4.09	7.80	11.51

그림 8.10 레즈넷 모델 구조

- 1개 입력 줄기 (input stem) + 4개 스테이지(stage)
 - 입력 줄기: 7x7 Conv + 배치 정규화 + ReLU + 3x3 최댓값 풀링
 - 스테이지: 여러 개의 잔차 블록

기본 블록

```
from torch import nn
```

```
class BasicBlock(nn.Module):
```

```
    expansion = 1
```

```
    def __init__(self, inplanes, planes, stride=1):
```

```
        super().__init__()
```

```
        self.conv1 = nn.Conv2d(
```

```
            inplanes, planes,
```

```
            kernel_size=3, stride=stride, padding=1, bias=False
```

```
        )
```

```
        self.bn1 = nn.BatchNorm2d(planes)
```

```
        self.relu = nn.ReLU(inplace=True)
```

```
        self.conv2 = nn.Conv2d(
```

```
            planes, planes,
```

```
            kernel_size=3, stride=1, padding=1, bias=False
```

```

)
self.bn2 = nn.BatchNorm2d(planes)

self.shortcut = nn.Sequential()
if stride != 1 or inplanes != self.expansion*planes:
    self.shortcut = nn.Sequential(
        nn.Conv2d(
            inplanes, self.expansion*planes,
            kernel_size=1, stride=stride, bias=False
        ),
        nn.BatchNorm2d(self.expansion*planes)
    )

def forward(self, x):
    out = self.conv1(x)
    out = self.bn1(out)
    out = self.relu(out)
    out = self.conv2(out)
    out = self.bn2(out)
    out += self.shortcut(x)
    out = self.relu(out)
    return out

```

- 합성곱 계층: 64→128→256→512 차원
- 입력 특징맵의 차원 수(inplanes), 출력 특징맵의 차원 수(planes)를 설정하는 매개변수 받아야 함
- 두 번째 스테이지~마지막 스테이지 첫 번째 블록의 첫 번째 합성곱 계층에만 stride=2로 사용하는 지점
 - 이미지의 크기 감소, 모델 깊이 증가 → 연산량 감소
 - 간격을 활용해 특징맵 크기를 감소
- 3×3 Conv, 배치 정규화, ReLU 다음 잔차 연결
 - 비어 있는 시퀀셜을 생성해 잔차 연결 변수로 사용
 - 입력값과 출력값의 차원이 동일하지 않을 경우를 고려해 W_s 구현
- 순방향 메소드에서 잔차 연결로 입력값과 출력값을 더하고 있음

병목 블록

```
class BottleneckBlock(nn.Module):
    expansion = 4

    def __init__(self, inplanes, planes, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(
            inplanes, planes,
            kernel_size=1, bias=False
        )
        self.bn1 = nn.BatchNorm2d(planes)
        self.conv2 = nn.Conv2d(
            planes, planes,
            kernel_size=3, stride=stride, padding=1, bias=False
        )
        self.bn2 = nn.BatchNorm2d(planes)
        self.conv3 = nn.Conv2d(
            planes, self.expansion*planes,
            kernel_size=1, bias=False
        )
        self.bn3 = nn.BatchNorm2d(self.expansion*planes)
        self.relu = nn.ReLU(inplace=True)

        self.shortcut = nn.Sequential()
        if stride != 1 or inplanes != self.expansion*planes:
            self.shortcut = nn.Sequential(
                nn.Conv2d(
                    inplanes, self.expansion*planes,
                    kernel_size=1, stride=stride, bias=False
                ),
                nn.BatchNorm2d(self.expansion*planes)
            )

    def forward(self, x):
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.relu(out)
```

```

out = self.conv2(out)
out = self.bn2(out)
out = self.relu(out)
out = self.conv3(out)
out = self.bn3(out)
out += self.shortcut(x)
out = self.relu(out)
return out

```

ResNet 모델

구조	출력 크기	ResNet-18	ResNet-34	ResNet-50	ResNet-101	ResNet-152
입력 줄기 (input stem)	112 × 112	7 × 7 64				
		stride=2, padding=3				
		7 × 7 MaxPool				
		stride=2, padding=1				
스테이지 1 (stage 1)	56 × 56	$\begin{bmatrix} 3 \times 3 & 64 \\ 3 \times 3 & 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3 & 64 \\ 3 \times 3 & 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 64 \\ 3 \times 3 & 64 \\ 1 \times 1 & 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 64 \\ 3 \times 3 & 64 \\ 1 \times 1 & 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 64 \\ 3 \times 3 & 64 \\ 1 \times 1 & 256 \end{bmatrix} \times 3$
스테이지 2 (stage 2)	28 × 28	$\begin{bmatrix} 3 \times 3 & 128 \\ 3 \times 3 & 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3 & 128 \\ 3 \times 3 & 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1 & 128 \\ 3 \times 3 & 128 \\ 1 \times 1 & 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1 & 128 \\ 3 \times 3 & 128 \\ 1 \times 1 & 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1 & 128 \\ 3 \times 3 & 128 \\ 1 \times 1 & 512 \end{bmatrix} \times 8$
스테이지 3 (stage 3)	14 × 14	$\begin{bmatrix} 3 \times 3 & 256 \\ 3 \times 3 & 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3 & 256 \\ 3 \times 3 & 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 & 256 \\ 3 \times 3 & 256 \\ 1 \times 1 & 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1 & 256 \\ 3 \times 3 & 256 \\ 1 \times 1 & 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1 & 256 \\ 3 \times 3 & 256 \\ 1 \times 1 & 1024 \end{bmatrix} \times 36$
스테이지 3 (stage 4)	7 × 7	$\begin{bmatrix} 3 \times 3 & 512 \\ 3 \times 3 & 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3 & 512 \\ 3 \times 3 & 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 512 \\ 3 \times 3 & 512 \\ 1 \times 1 & 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 512 \\ 3 \times 3 & 512 \\ 1 \times 1 & 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1 & 512 \\ 3 \times 3 & 512 \\ 1 \times 1 & 2048 \end{bmatrix} \times 3$
완전 연결 계층 (FC Layer)	1 × 1	1 × 1 AvgPool 1000 dimension(class)				
Params		11,689,512	21,797,672	25,557,032	44,549,160	60,192,808
GFLOPS		1.81	3.66	4.09	7.80	11.51

그림 8.10 레즈넷 모델 구조

Grad-CAM (Gradient-weighted Class Activation Mapping)

- 설명 가능한 인공지능(eXplainable Artificial Intelligence, XAI) 기술 중 하나
 - XAI: 인공지능 모델이 도출한 결과를 인간이 이해하고 검증할 수 있게 하는 방법
- 딥러닝 모델의 내부 동작 원리를 시각화하는 방법
- 딥러닝 모델이 이미지 데이터로 예측할 때 **블랙박스 문제** 유발
 - 수백만 개 매개변수를 갱신하고 비선형 연산을 통해 학습하기 때문

클래스 활성화 맵 (Class Activation Map, CAM)

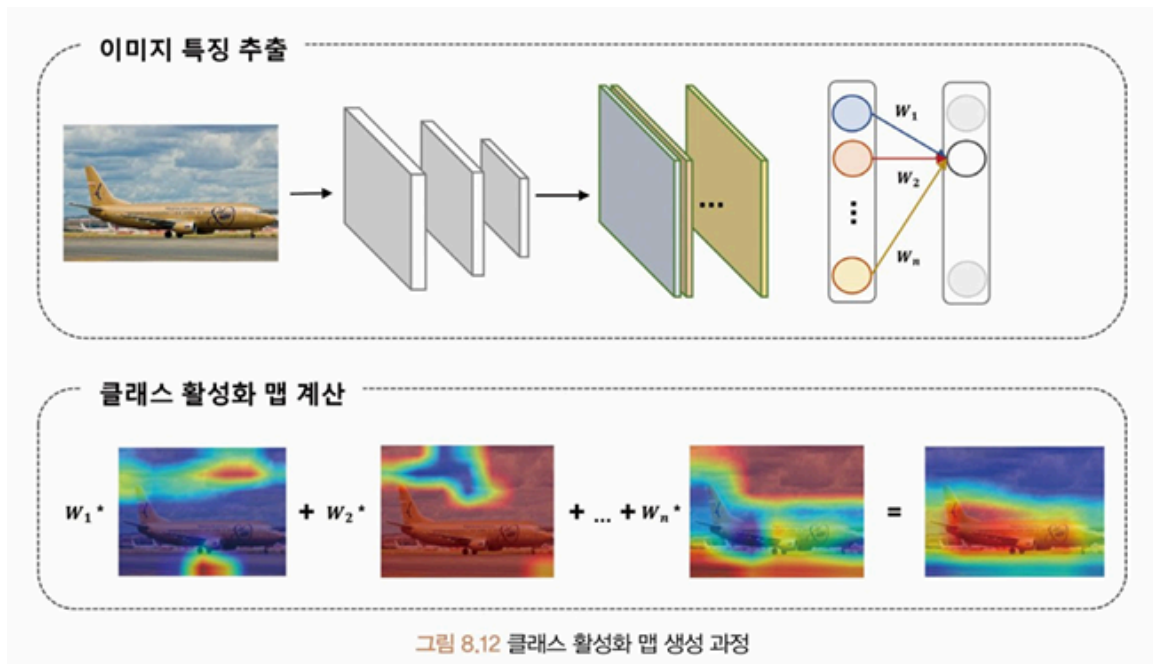
- 딥러닝 모델에서 특정 클래스와 관련된 입력 이미지 영역을 시각화하는 방법
- 주로 cnn에서 활용
- 특징 맵을 활용해 입력 이미지의 어떤 부분이 특정 클래스와 관련 있는지 시각화
 - 모델이 각 이미지 영역에 주목하는 정도와 영향도 표현



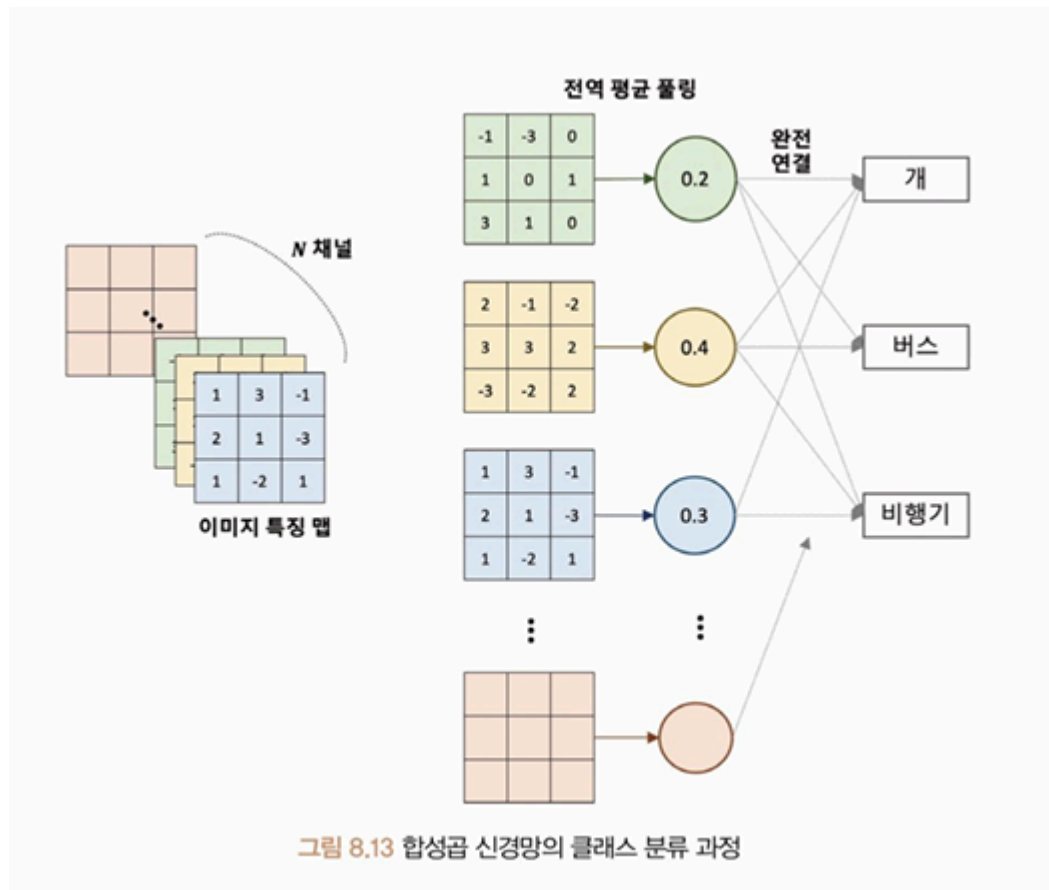
그림 8.11 클래스 활성화 맵을 히트 맵으로 시각화

클래스 활성화 맵 생성 방식

- 합성곱 신경망의 특징맵 채널과 분류기의 매개변수 값을 활용해 생성
- 특징 맵: 각 채널 분류기의 가중치를 곱해 채널마다 가중치 합 계산
 - 정규화로 어느 영역에서 가장 강하게 활성화되는지 수치화
 - 시각화



- 클래스 활성화 맵 생성 과정
 - 합성곱 신경망의 마지막 합성곱 계층에서 전역 평균 풀링(GAP) 적용 → 각 픽셀 정보를 1차원으로 펴고 출력으로 사용
 - 특징 맵의 채널별로 특징값 통합, 이 값을 분류기에 전달
 - 분류기 매개변수가 출력 클래스에 대한 특징값 가중치가 됨
 - 가중치 높음 → 분류에 영향 높음



- 가중치를 특징맵에 전달하는 방법

수식 8.3 클래스 활성화 맵 생성 수식

$$L^c(i, j) = \sum_k w_k^c f_k(i, j)$$

- L^c : 클래스 c 에 대한 클래스 활성화 맵 출력값
- i, j : CAM 행렬의 행, 열
- w : 클래스 c 에 대한 채널의 특징 맵 분류 가중치
- f : 특징맵 k 번째 채널의 i 행 j 열 값 (마지막 conv 활성화 값)

클래스 활성화 맵 적용

Grad-CAM

- GAP 계층 사용 X, 마지막 합성곱 계층에서 역전파로 중요도 계산

- CAM 단점: 전역 평균 풀링 계층을 포함해야 함
 - GAP 사용 안 하는 합성곱 신경망이면 모델 구조 변경하고 재학습해야 함
 - 범용성 X, 시각화를 위해 많은 리소스 소모
- Grad-CAM 생성 방식

수식 8.4 Grad-CAM 생성 수식

$$L^c(i,j) = ReLU\left(\sum_k a_k^c f_k(i,j)\right)$$

- a: 클래스 c에 대한 k 채널의 특징 맵 가중치 (마지막 conv 가중치)

수식 8.5 특징 맵 가중치

$$a_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial y^c}{\partial f_k(i,j)}$$

- y: 클래스 c에 대한 점수
 - f_k: 특징 맵 k번째 채널의 i행 j열 값
- ⇒ y가 f에 얼마나 민감하게 반응하느냐에 대한 수식
- ⇒ 역전파를 통한 기울기를 계산하는 연산
- Z: 특징 맵 픽셀 수

Grad-CAM 적용